

AI Lesson Coach — Onboarding Guide

Workplace Edge / LevelUP

May 8, 2026

CONTENTS

AI Lesson Coach — Onboarding Guide	
Table of Contents	
1. What is this product?	
2. Who is it for?	
3. The two audiences (learner + admin)	
Learner experience	
Admin experience	
4. Feature tour	
5. How a chat actually works (end-to-end)	
6. Architecture at a glance	
7. Repository layout	
8. Environment variables (.env)	
Template (from .env.example)	
Variable reference	
Security checklist for the .env file	
9. Local setup (step-by-step)	
Prerequisites	
Steps	
"It's broken" troubleshooting	
10. Day-to-day developer workflows	
Frontend	
Backend	
Working through GSD (per CLAUDE.md)	
11. Database & migrations	
Schema models	
Migration commands (from /server)	

12. Deployment

Production environment variables

CORS in production

CSP / iframe embedding

13. Safety, privacy & compliance

Layered safety pipeline (every chat request)

Data retention

Tenancy caveat

14. Glossary

15. Who to ask for help

Useful external links

AI Lesson Coach — Onboarding Guide

Workplace Edge | LevelUP AI Chatbot

CONFIDENTIAL — INTERNAL USE ONLY

This document contains environment variable templates and infrastructure details. Do **not** share externally, post in public Slack channels, attach to support tickets, or upload to third-party services. If shared with a contractor or new hire, deliver it through a trusted channel (1Password, signed email) and rotate any credentials they no longer need access to when they leave.

Table of Contents

1. What is this product?
 2. Who is it for?
 3. The two audiences (learner + admin)
 4. Feature tour
 5. How a chat actually works (end-to-end)
 6. Architecture at a glance
 7. Repository layout
 8. Environment variables (`.env`)
 9. Local setup (step-by-step)
 10. Day-to-day developer workflows
 11. Database & migrations
 12. Deployment
 13. Safety, privacy & compliance
 14. Glossary
 15. Who to ask for help
-

1. What is this product?

AI Lesson Coach is a chat-based training tool for **Workplace Edge**, a communication-skills program. It helps learners practice difficult workplace conversations — giving feedback, setting boundaries, pushing back on a deadline, clarifying tasks — by talking to an AI coach.

The key idea: the AI is **grounded** in lesson documents that an admin uploads. It is not a general-purpose chatbot. If you ask it about the weather or to write code, it will redirect you back to the lesson material. This makes it safe to deploy inside a learning platform without worrying that learners will get unrelated or unsafe content.

The product ships in two forms:

- A **standalone web app** at `/chat` (learner) and `/admin` (instructor).
 - An **embedded chat widget** that can be dropped into LearnWorlds course pages via an iframe.
-

2. Who is it for?

Audience	What they do	Where they spend time
Learner	Practices a workplace conversation skill by chatting with the AI coach. May role-play with the AI as a difficult coworker.	<code>/chat</code> , embedded widget inside a LearnWorlds course
Admin / Instructor	Uploads <code>.docx</code> lesson documents that ground the AI's answers. Reviews analytics on what learners are asking.	<code>/admin</code>
Developer	Maintains the codebase, fixes bugs, ships features.	This repo
Stakeholder / PM	Reviews analytics, decides product direction.	<code>/admin</code> analytics dashboard

3. The two audiences (learner + admin)

LEARNER EXPERIENCE

1. Lands on `/chat` (or sees the widget on a LearnWorlds page).
2. Sees a welcome screen with topic cards: *Give Feedback, Set a Boundary, Push Back on Deadline, Clarify Tasks, General Coaching*.
3. Picks a topic, types a question (e.g. “How do I tell my manager I can’t hit Friday’s deadline?”).
4. The AI replies in a three-beat coaching rhythm:
 1. **Listens & acknowledges** what the learner is dealing with.
 2. **Teaches** the relevant skill drawn from the uploaded lesson.
 3. **Suggests a way to practice** the skill this week.
5. Below the answer, a **follow-up suggestion chip** appears as a clickable button to go deeper.
6. The learner can click **Practice This** to enter **role-play mode** — the AI plays a difficult coworker for two turns, then automatically gives coaching feedback on how the learner did.
7. Each AI message has thumbs-up / thumbs-down buttons for quick rating.

ADMIN EXPERIENCE

1. Goes to `/admin` . The page is locked behind an admin token.
2. Pastes the `ADMIN_API_TOKEN` value to unlock. Token is stored in `sessionStorage` (cleared when the browser tab closes or **Lock Admin** is clicked).
3. Drags a `.docx` lesson file onto the upload zone. The system extracts plain text using **Mammoth** and stores it as the AI’s grounding context.
4. Manages the lesson library — view, delete, re-upload.
5. Reviews **analytics**:
 - Total sessions, average messages per session, total ratings, satisfaction %.
 - Bar chart of which **intents** learners ask about most.
 - Bar chart of which **lessons** are referenced most.

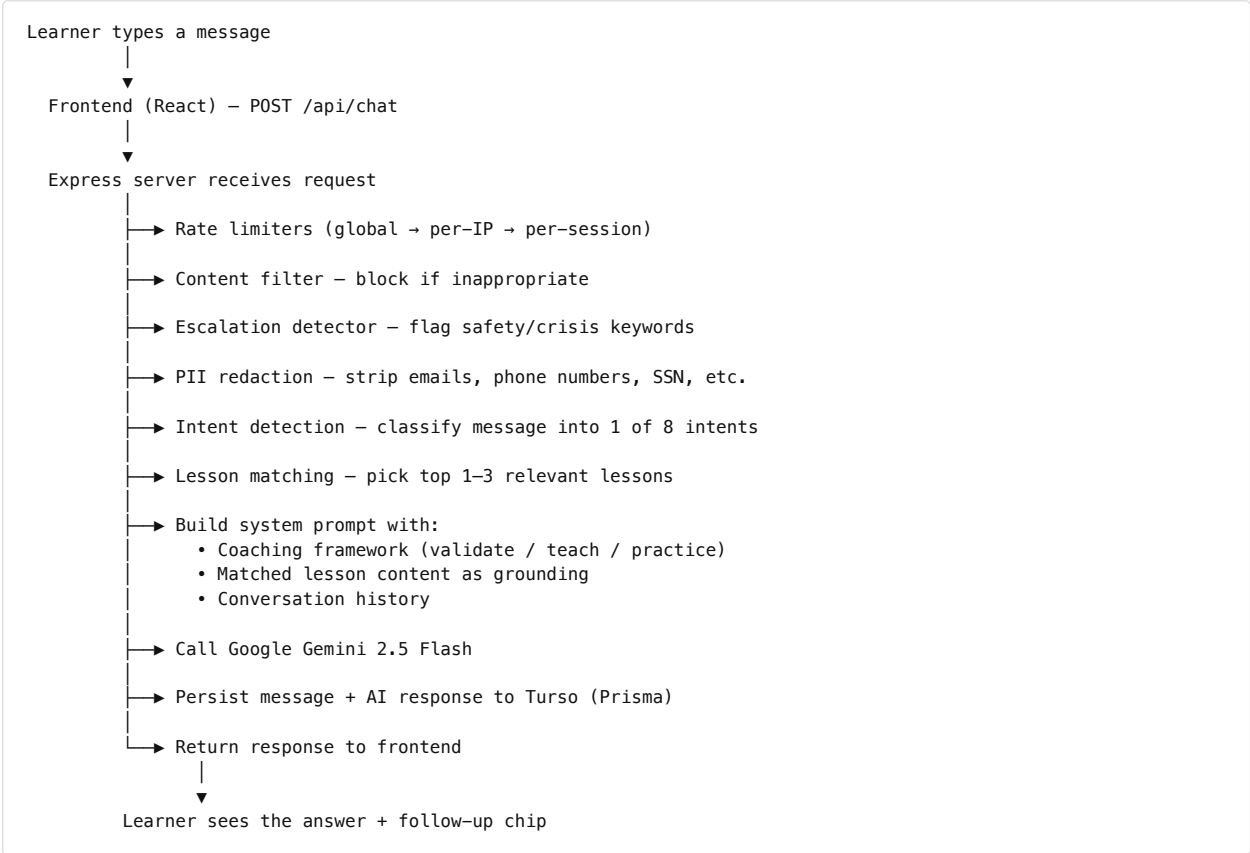
4. Feature tour

Feature	What it does	Where in code
Intent detection	Classifies each learner message into one of 8 intents (give feedback, set boundary, push back deadline, clarify tasks, general coaching, lesson-specific, general, off-topic) using keyword + phrase rules. Drives which lesson content is sent to the LLM.	<code>server/src/lib/intent.ts</code>
Lesson matching	Scores each uploaded lesson against the learner's message and picks the most relevant 1–3 to send as grounding context.	<code>server/src/lib/intent.ts</code> (<code>scoreLessonMatch</code>)
Three-beat coaching structure	The AI's responses follow a validate → teach → suggest-practice rhythm. The framework is intentionally never named to the learner.	<code>server/src/lib/gemini.ts</code> (system prompt)
Practice / role-play mode	A 2-turn role-play where the AI plays a difficult coworker, followed by automatic coaching feedback.	<code>server/src/routes/chat.ts</code> + <code>RolePlayToggle.tsx</code>
Follow-up suggestion chips	Parsed from the AI response, surfaced as clickable buttons that auto-send the follow-up question.	<code>src/components/ChatMessage.tsx</code>
Thumbs up/down rating	Per-message rating, stored in the <code>Rating</code> table and aggregated for analytics.	<code>src/components/ChatMessage.tsx</code> + <code>POST /api/rating</code>
Content safety filter	Blocks inappropriate content before it reaches the LLM.	<code>server/src/lib/contentFilter.ts</code>
Escalation detection	Detects safety/crisis keywords (harassment, self-harm, legal). Prepends a banner to the response and logs the event.	<code>server/src/lib/escalation.ts</code> + <code>EscalationBanner.tsx</code>
PII redaction	Strips personal data (emails, phone numbers, SSN/SIN, credit card numbers) before storing messages in the database.	<code>server/src/lib/privacy.ts</code>
HR disclaimer	Reminds learners this is a learning tool, not a substitute for HR. Shown in the UI and in the system prompt.	UI + <code>server/src/lib/gemini.ts</code>
Admin authentication	Bearer-token auth on admin routes. Token stored in <code>sessionStorage</code> on the client.	<code>server/src/middleware/auth.ts</code>
Rate limiting	Multi-layer: <code>globalRateLimiter</code> (200/min/IP) → <code>chatLimiter</code> (30/min/IP) → <code>chatSessionLimiter</code> (100/min/session). Upload route: 5/min/IP.	<code>server/src/middleware/rateLimiter.ts</code>
Lesson cache	Caches uploaded lesson content in memory to avoid hitting the DB on every chat request.	<code>server/src/lib/lessonCache.ts</code>
Vocabulary extraction	Parses glossary terms from uploaded lessons and exposes them to the frontend.	<code>server/src/lib/vocabulary.ts</code> , <code>vocabularyPdf.ts</code>
Handout suggestions	Lessons can carry worksheet references (<code>{ title, url? }</code>) parsed at upload and surfaced contextually.	<code>server/src/lib/handouts.ts</code>
SCORM / iframe embed	<code>/embed</code> route is a minimal-chrome chat for embedding in LMS course pages.	<code>src/pages/EmbedChatPage.tsx</code>
Collapsible widget	<code>/widget</code> route renders as a floating chat button that expands.	<code>src/pages/WidgetPage.tsx</code> , <code>ChatWidget.tsx</code>
i18n framework	<code>react-i18next</code> is wired up with English locale; structure ready for additional languages.	<code>src/i18n.ts</code> , <code>src/locales/en.json</code>

Feature	What it does	Where in code
Data retention	Sessions older than 30 days are eligible for cleanup via <code>POST /api/cleanup</code> (admin-only).	<code>server/src/routes/cleanup.ts</code>

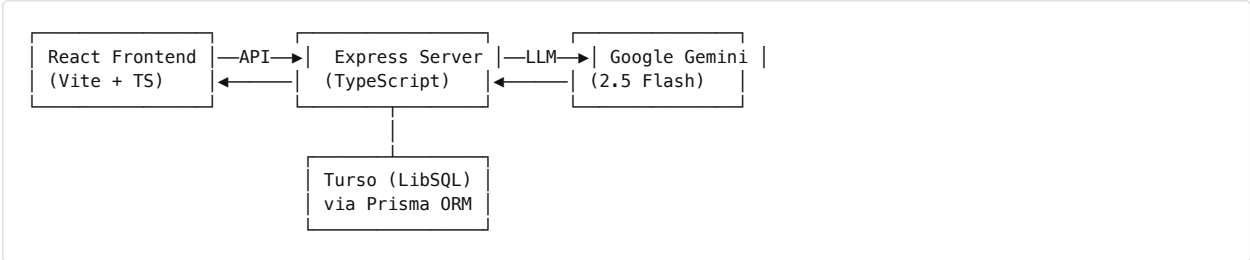
5. How a chat actually works (end-to-end)

This is the path a single learner message takes through the system. Useful for non-technical readers who want to understand “what does the AI actually do?”



The most important detail: the AI is *only* allowed to draw from uploaded lesson content. The system prompt explicitly tells it to re-direct off-topic questions back to the lesson material. This is the safety guarantee that lets us ship to learners.

6. Architecture at a glance



Two independent packages in one repo:

Package	Role	Key tech
Root (/)	React 18 frontend	Vite, TypeScript, Tailwind CSS, react-router-dom, react-i18next, axios
/server	Express API	TypeScript, tsx (dev runtime), Prisma, multer, mammoth, helmet, express-rate-limit

External services:

- **Google Gemini 2.5 Flash** — the LLM that generates coach responses. Accessed via `@google/generative-ai`.
- **Turso (LibSQL)** — managed SQLite database. The runtime DB for production.
- **Vercel** — hosts the frontend (SPA via `vercel.json` rewrite).
- **Railway** (or similar Node host) — hosts the Express server.
- **LearnWorlds** — the LMS where the chat is embedded as an iframe widget.

7. Repository layout

```

ai-chatbot/
├── src/                                # FRONTEND
│   ├── main.tsx                       # React entry point
│   ├── App.tsx                        # Router (/chat, /admin, /embed, /widget)
│   ├── pages/
│   │   ├── ChatPage.tsx              # Learner chat
│   │   ├── AdminPage.tsx            # Admin console
│   │   ├── EmbedChatPage.tsx        # Iframe-friendly chat
│   │   └── WidgetPage.tsx           # Floating widget
│   ├── components/
│   │   ├── ChatExperience.tsx
│   │   ├── ChatMessage.tsx
│   │   ├── ChatInput.tsx
│   │   ├── ChatWidget.tsx
│   │   ├── EscalationBanner.tsx
│   │   ├── LessonsTable.tsx
│   │   ├── RolePlayToggle.tsx
│   │   ├── AnalyticsDashboard.tsx
│   │   ├── UploadZone.tsx
│   │   ├── VocabularyDownloads.tsx
│   │   ├── TypingIndicator.tsx
│   │   └── Toast.tsx
│   ├── api.ts                        # Axios client
│   ├── types.ts                      # TypeScript interfaces
│   ├── i18n.ts                       # i18n config
│   └── locales/en.json               # English strings
├── server/                           # BACKEND
│   ├── src/
│   │   ├── index.ts                 # Express bootstrap
│   │   ├── env.ts                   # Loads .env from repo root
│   │   ├── routes/
│   │   │   ├── chat.ts              # POST /api/chat (the main flow)
│   │   │   ├── upload.ts            # POST /api/upload
│   │   │   ├── lessons.ts           # GET / DELETE lessons
│   │   │   ├── rating.ts            # POST rating
│   │   │   ├── analytics.ts         # GET analytics
│   │   │   ├── cleanup.ts           # POST cleanup (admin)
│   │   │   ├── admin.ts             # GET admin/session
│   │   │   ├── health.ts            # GET health (no auth)
│   │   │   └── ipDebug.ts           # Dev-only IP check
│   │   ├── middleware/
│   │   │   ├── auth.ts              # requireAdmin (bearer token)
│   │   │   └── rateLimiter.ts       # Multi-tier rate limiting
│   │   └── lib/
│   │       ├── gemini.ts            # LLM client + system prompt
│   │       ├── intent.ts            # Intent detection
│   │       ├── complexity.ts         # Simple vs complex prompts
│   │       ├── breadth.ts           # Broad vs focused questions
│   │       ├── escalation.ts         # Safety/crisis detection
│   │       ├── contentFilter.ts      # Inappropriate content guard
│   │       ├── privacy.ts           # PII redaction + retention
│   │       ├── handouts.ts          # Worksheet references
│   │       ├── vocabulary.ts        # Glossary parsing
│   │       ├── lessonCache.ts       # In-memory lesson cache
│   │       ├── lessonCleanup.ts     # Lesson lifecycle
│   │       ├── sanitize.ts
│   │       ├── db.ts                # Prisma + LibSQL adapter
│   │       └── env.ts
│   ├── prisma/
│   │   ├── schema.prisma            # DB schema
│   │   └── migrations/              # Migration history
│   └── scripts/
│       └── applyTursoMigrations.ts   # Manual deploy helper
├── docs/                             # Existing docs (USER_GUIDE, ADMIN_GUIDE, etc.)
├── .env.example                       # Environment variable template
└── .env                              # Real values – NEVER COMMITTED (in .gitignore)

```

```
├─ package.json          # Frontend deps
├─ server/package.json   # Backend deps
├─ vercel.json           # SPA rewrite config
├─ tailwind.config.ts
├─ vite.config.ts
└─ README.md
```

8. Environment variables (`.env`)

The server loads a single `.env` file from the **repo root** — both frontend (Vite) and backend (Express) read from the same file. The frontend uses any `VITE_*` prefixed variable at build time; the backend uses everything else at runtime.

⚠️ **The real `.env` is NOT in this repo** — it is `.gitignored`. New hires must create their own from the template below and request the actual secret values from a team lead. **Never paste real secret values into Slack, email, or a ticket.** Use 1Password or another secret manager.

TEMPLATE (FROM `.ENV.EXAMPLE`)

```
# Frontend (Vite)
VITE_API_URL=http://localhost:3000

# Backend (Express loads this file from ../.env relative to server/)
# The running server connects to Turso via these two variables.
# `npm run db:migrate:deploy` (from /server) applies pending migrations to
# whatever URL is set here — double-check before running against prod.
TURSO_DATABASE_URL=
TURSO_AUTH_TOKEN=
GEMINI_API_KEY=

# Used by the Prisma CLI only (db:push:local, db:migrate:dev). Defaults to
# a local SQLite file when unset; uncomment to override.
# PRISMA_DATABASE_URL=file:./dev.db

# Optional: comma-separated extra CORS origins (e.g. custom domain).
# Vercel *.vercel.app is allowed by default in code.
# ALLOWED_ORIGINS=https://coach.example.com
```

VARIABLE REFERENCE

Variable	Required	What it is	How to get it
VITE_API_URL	✓	Base URL the frontend hits for API calls. Locally <code>http://localhost:3000</code> ; in prod the Railway server URL.	Set per-environment. Local default works out of the box.
TURSO_DATABASE_URL	✓	The <code>libsql://...turso.io</code> URL of the Turso database the server reads/writes.	Turso dashboard → your DB → Database URL . Ask the team lead for the prod URL.
TURSO_AUTH_TOKEN	✓	JWT that authenticates the server to Turso.	Turso dashboard → your DB → Tokens → create read-write token.
GEMINI_API_KEY	✓	API key for Google Gemini 2.5 Flash.	aistudio.google.com → Get API key . Free tier is fine for local dev.
ADMIN_API_TOKEN	✓	Shared bearer token that unlocks <code>/admin</code> and admin-only API routes.	Generate a long random string (e.g. <code>openssl rand -hex 32</code>). Same value goes in the server <code>.env</code> and is pasted into the admin UI to unlock.
PRISMA_DATABASE_URL	✗	Used by the Prisma CLI only (for <code>db:migrate:dev</code>). Defaults to <code>file:./dev.db</code> — a local SQLite file — when unset.	Leave unset for local dev. Never point this at production.
ALLOWED_ORIGINS	✗	Comma-separated extra origins to allow in CORS, on top of <code>localhost:5173</code> (always allowed in dev) and the Vercel deploy URL.	Add custom domains here: <code>https://coach.workplaceedge.com,https://learn.example.com</code> .
PORT	✗	Express listen port.	Defaults to <code>3000</code> . Railway sets this automatically.
NODE_ENV	✗	<code>production</code> switches stricter CSP frame-ancestors and CORS rules.	Set to <code>production</code> in deployment, leave unset locally.
VERCEL_PROD_URL	✗	Allowed iframe ancestor in production (the Vercel deployment URL).	Set in Railway env to your Vercel domain.

SECURITY CHECKLIST FOR THE `.ENV` FILE

- ✓ `.env` is in `.gitignore` (line 6) — confirm before any new commit.
- ✓ `git log -- .env` should return **nothing**. If it doesn't, rotate every secret immediately.
- ✓ Never paste secrets into Slack, email, GitHub issues, or PR descriptions.
- ✓ Rotate `GEMINI_API_KEY`, `TURSO_AUTH_TOKEN`, and `ADMIN_API_TOKEN` whenever a teammate with access leaves.
- ✓ The `ADMIN_API_TOKEN` should be at least 32 random bytes (`openssl rand -hex 32`).

9. Local setup (step-by-step)

This works on macOS, Linux, and WSL on Windows.

PREREQUISITES

- **Node.js 18+** (nodejs.org) — confirm with `node -v`.
- **npm** (ships with Node) — confirm with `npm -v`.
- **A code editor** — VS Code is the team standard.
- **A terminal** — iTerm2, Terminal.app, or VS Code's built-in.

STEPS

1. Clone the repo.

```
git clone https://github.com/Gurehmat/AI-Chatbot.git
cd AI-Chatbot
```

2. Create your `.env`.

```
cp .env.example .env
```

Open `.env` in your editor and fill in the four required values (`TURSO_DATABASE_URL` , `TURSO_AUTH_TOKEN` , `GEMINI_API_KEY` , `ADMIN_API_TOKEN`). Ask a team lead for the dev Turso credentials, or create your own free Turso DB at turso.tech.

3. Install frontend dependencies.

```
npm install
```

(Run from the repo root.)

4. Install server dependencies.

```
cd server
npm install
```

The `postinstall` hook runs `prisma generate` automatically — you'll see Prisma client output.

5. Apply database migrations (only the first time, or after schema changes).

```
# Still in /server
npm run db:migrate:deploy
```

⚠ This writes to whatever `TURSO_DATABASE_URL` is in your `.env`. Confirm the URL before running.

6. Start the dev servers in two terminals.

Terminal 1 — frontend (port 5173):

```
# From repo root
npm run dev
```

Terminal 2 — backend (port 3000):

```
cd server
npm run dev
```

7. Open the app.

- Learner chat: http://localhost:5173/chatSMARTPANTS_PRESERVED_955SMARTPANTS_PRESERVED_956
- Admin: http://localhost:5173/adminSMARTPANTS_PRESERVED_959 (unlock with your `ADMIN_API_TOKEN`)
- Widget: http://localhost:5173/widgetSMARTPANTS_PRESERVED_963SMARTPANTS_PRESERVED_964
- Embed: http://localhost:5173/embedSMARTPANTS_PRESERVED_967SMARTPANTS_PRESERVED_968

"IT'S BROKEN" TROUBLESHOOTING

<u>Symptom</u>	<u>Likely cause</u>	<u>Fix</u>
GEMINI_API_KEY is re-quired on server start	<code>.env</code> missing or in the wrong place	<code>.env</code> must be at the repo root, not inside <code>/server</code>
Frontend loads but every chat re- turns an error	<code>VITE_API_URL</code> doesn't match the server URL	Set <code>VITE_API_URL=http://localhost:3000</code> and restart <code>npm run dev</code>
<code>/admin</code> keeps re-locking	<code>ADMIN_API_TOKEN</code> mismatch be- tween client and server	Server must have the same token in <code>.env</code> as you paste in the UI
<code>prisma generate</code> fails	Wrong Node version	Use Node 18 or newer: <code>nvm use 20</code>
Migrations fail with libSQL errors	<code>TURSO_AUTH_TOKEN</code> is invalid or expired	Generate a fresh token in the Turso dashboard
CORS error in the browser console	Origin not allowed	Add to <code>ALLOWED_ORIGINS</code> env var

10. Day-to-day developer workflows**FRONTEND**

```
npm run dev      # Vite dev server, hot reload on :5173
npm run build    # Production build → dist/
npm run preview  # Preview production build locally
```

BACKEND

```
cd server
npm run dev      # tsx watch mode on :3000
npm run build    # tsc → dist/
npm test        # Vitest test suite
npm run test:watch # Watch mode
```

WORKING THROUGH GSD (PER CLAUDE.MD)

The team uses a structured planning workflow called **GSD**. Before making code changes:

- `/gsd-quick` for small fixes and ad-hoc tasks
- `/gsd-debug` for investigations and bug fixes
- `/gsd-execute-phase` for planned phase work

Direct edits outside of GSD are discouraged unless the user explicitly asks to bypass it.

11. Database & migrations

Database: Turso (LibSQL) — managed SQLite over HTTP — accessed via Prisma ORM with the `@prisma/adapters-libsql` adapter.

SCHEMA MODELS

Model	Purpose
<code>Session</code>	One row per chat conversation. Links to messages and intent logs.
<code>Message</code>	Each user/assistant turn. PII-redacted before insert. Linked to ratings.
<code>Rating</code>	Thumbs up/down per AI message. Powers the satisfaction metric.
<code>IntentLog</code>	Records which intent + which lessons were used for each user turn. Powers the intent-distribution chart.
<code>Lesson</code>	Uploaded <code>.docx</code> content as plain text + parsed handouts JSON.

MIGRATION COMMANDS (FROM `/SERVER`)

Command	What it does	When to use
<code>npm run db:migrate:dev -- --name <change></code>	Author a new migration locally against <code>PRISMA_DATABASE_URL</code> (defaults to <code>file:./dev.db</code>). Generates a folder under <code>prisma/migrations/</code> .	After editing <code>schema.prisma</code> .
<code>npm run db:migrate:deploy</code>	Apply pending committed migrations to whatever <code>TURSO_DATABASE_URL</code> is in <code>.env</code> . Tracks state in <code>_prisma_migrations</code> .	First-time setup, or after pulling new migrations from main.
<code>npm run db:migrate:deploy -- --dry</code>	Print pending migrations without applying.	Sanity-check before deploying.
<code>npm run db:push:local</code>	Push schema directly to local SQLite without generating a migration.	Fast iteration only — never against prod.

Why a custom deploy script? Prisma's native `migrate deploy` doesn't yet support libSQL over HTTP. `server/scripts/apply-TursoMigrations.ts` uses `@libsql/client` to apply migrations and tracks state in a `_prisma_migrations` table that's schema-compatible with Prisma's, so we can swap to native `migrate deploy` if/when it lands.

12. Deployment

Component	Host	How
Frontend	Vercel	<code>vercel.json</code> handles SPA routing. Push to main → Vercel rebuilds. Env vars set in the Vercel project dashboard at build time (<code>VITE_API_URL</code>).
Backend	Railway (or any Node host)	Builds with <code>npm run build</code> , runs <code>node dist/index.js</code> . Env vars set in Railway secrets.
Database	Turso	Managed; no deploy step. Schema changes via <code>npm run db:migrate:deploy</code> .
LLM	Google Gemini API	No deploy; just an API key.

PRODUCTION ENVIRONMENT VARIABLES

The server in production needs **all** of:

- `TURSO_DATABASE_URL` (production Turso URL)
- `TURSO_AUTH_TOKEN` (production token)
- `GEMINI_API_KEY` (the same or a separate prod key)
- `ADMIN_API_TOKEN` (different from dev)
- `ALLOWED_ORIGINS` (the Vercel prod URL + any custom domains)
- `VERCEL_PROD_URL` (for the helmet CSP frame-ancestors directive)
- `NODE_ENV=production`

CORS IN PRODUCTION

Production CORS is **strict allowlist** — no wildcard. Origins:

- Always: `localhost:5173`, `127.0.0.1:5173` (dev only — denied in prod via `NODE_ENV` check)
- Production: whatever is in `ALLOWED_ORIGINS`

The `/api/health` endpoint is mounted **before** CORS so uptime monitors making no-Origin requests can hit it.

CSP / IFRAME EMBEDDING

`helmet` enforces a Content Security Policy. The `frame-ancestors` directive controls who can embed the app in an iframe:

- Production: `https://*.learnworlds.com` + `VERCEL_PROD_URL`
- Dev: `'self'` + `http://localhost:5173` + `http://localhost:*`

If embedding fails on a new LearnWorlds school subdomain, this is the first place to look.

13. Safety, privacy & compliance

This is a learner-facing product touching workplace topics — get this section right.

LAYERED SAFETY PIPELINE (EVERY CHAT REQUEST)

1. **Rate limit** — global → per-IP → per-session. Stops floods.
2. **Content filter** (`contentFilter.ts`) — blocks inappropriate input *before* it reaches the LLM.
3. **Escalation detection** (`escalation.ts`) — flags harassment, self-harm, and legal-distress keywords. Returns a banner shown in the UI, logs the event, and never silences the response — the AI still answers, but with a safety prefix.
4. **PII redaction** (`privacy.ts`) — strips emails, phone numbers, SSN/SIN, and credit-card-shaped numbers from the message **before** it is stored. Storage is never raw input.
5. **Grounded responses** — the system prompt forces the AI to answer only from uploaded lesson material. Off-topic questions get a polite redirect.
6. **HR disclaimer** — UI and system prompt remind learners this is a learning tool, not HR.

DATA RETENTION

- Sessions older than **30 days** are eligible for cleanup.
- `POST /api/cleanup` (admin-only) runs `cleanExpiredSessions()` from `lib/privacy.ts`.
- No automated cron in v1.1 — admin runs it manually or via a scheduled CI job.

TENANCY CAVEAT

Multiple LearnWorlds tenants may share an egress IP. The 30/min/IP rate limit on `/api/chat` could lock out a whole organization in extreme cases. This is documented; if it bites, pair with a per-session bucket.

14. Glossary

Term	Meaning
Workplace Edge	The brand / customer this product was built for. A communication-skills training program.
Learner	The end user — someone practicing communication skills.
Admin / Instructor	The person who uploads lesson materials and reviews analytics.
Intent	The kind of help the learner is asking for. One of 8 categories detected by keyword rules.
Lesson	A <code>.docx</code> file uploaded by an admin, used as grounding context for the AI.
Grounding	Constraining an LLM to only answer from a specific document. The opposite of free-form chat.
Three-beat coaching	The internal scaffolding of every AI response: validate → teach → suggest practice. Never named to the learner.
Practice mode / Role-play	A 2-turn exchange where the AI plays a difficult coworker, followed by automatic coaching feedback.
PII	Personally Identifiable Information — emails, phone numbers, SSN, etc. Stripped before storage.
Escalation	A safety-critical message (harassment, self-harm). Triggers a banner and logs the event.
Handout	A worksheet reference (<code>{ title, url? }</code>) parsed from a lesson and surfaced contextually.
SCORM	An LMS standard. The <code>/embed</code> route supports SCORM-style iframe embedding.
LearnWorlds	The LMS that hosts the embedded chat widget for the production deployment.
Turso	Managed SQLite database accessed over HTTP via libSQL. The runtime DB.
LibSQL	The fork of SQLite that Turso uses. Same SQL, different transport.
Prisma	The ORM (object-relational mapper) used to talk to the DB from TypeScript.
Vite	The frontend build tool. Replaces webpack. Fast hot reload.
tsx	A TypeScript runner (like <code>node</code> for TS). Used by the server in dev mode.
Mammoth	Library that extracts plain text from <code>.docx</code> files.
GSD	The team's structured planning workflow (Get Stuff Done). See <code>.planning/</code>.
CSP	Content Security Policy — browser-level rules about what scripts/iframes are allowed.

15. Who to ask for help

If you're stuck on...	Ask...
Getting <code>.env</code> values	A current team lead — values are in 1Password / shared secrets vault.
AI behavior / coaching prompt tuning	Whoever owns <code>server/src/lib/gemini.ts</code>.
Database schema or migrations	Whoever last touched <code>server/prisma/schema.prisma</code>. Check <code>git log</code>.
Deployment / infra	The person who owns the Railway and Vercel projects.
Product direction / scope	The PM / Workplace Edge stakeholder.
The codebase in general	Daniel Chahine (chahinedaniel0@gmail.com).

USEFUL EXTERNAL LINKS

- [Repo: https://github.com/Gurehmat/AI-ChatbotSMARTPANTS_PRESERVED_1563SMARTPANTS_PRESERVED_1564](https://github.com/Gurehmat/AI-ChatbotSMARTPANTS_PRESERVED_1563SMARTPANTS_PRESERVED_1564)
- [Google AI Studio \(Gemini keys\): https://aistudio.google.comSMARTPANTS_PRESERVED_1567SMARTPANTS_PRESERVED_1568](https://aistudio.google.comSMARTPANTS_PRESERVED_1567SMARTPANTS_PRESERVED_1568)
- [Turso dashboard: https://turso.techSMARTPANTS_PRESERVED_1571SMARTPANTS_PRESERVED_1572](https://turso.techSMARTPANTS_PRESERVED_1571SMARTPANTS_PRESERVED_1572)
- [Vercel dashboard: https://vercel.comSMARTPANTS_PRESERVED_1575SMARTPANTS_PRESERVED_1576](https://vercel.comSMARTPANTS_PRESERVED_1575SMARTPANTS_PRESERVED_1576)
- [LearnWorlds: https://www.learnworlds.comSMARTPANTS_PRESERVED_1579SMARTPANTS_PRESERVED_1580](https://www.learnworlds.comSMARTPANTS_PRESERVED_1579SMARTPANTS_PRESERVED_1580)

[Last updated: 2026-05-08 · This document covers v1.1 of the AI Lesson Coach.](#)